

Лабораторна робота №4

Тема: Тестування та CI

Мета роботи: навчитися писати тести для NextJS-додатків.

Практичні завдання

Завдання 1: Налаштуйте фреймворк для юніт-тестів

На першому етапі було налаштовано середовище для модульного тестування за допомогою фреймворку Jest та бібліотеки React Testing Library. У програмній частині створено конфігураційний файл `jest.config.ts`, де встановлено середовище `jsdom` для симуляції браузерного оточення та інтегровано `jest.setup.ts` для розширення матчерів DOM. Згідно з вимогами, було активовано механізм автоматичного збору статистики (`collectCoverage`) та встановлено жорсткий мінімальний поріг покриття (`coverage threshold`) на рівні 40% для гілок, функцій та рядків коду (рис. 1). Тестовий запуск підтвердив коректність налаштувань: система автоматично аналізує всі файли в директорії `src` та блокує виконання при недостатньому обсязі протестованого функціоналу, що забезпечує стабільний контроль якості розробки (рис. 2).

```
jest.config.ts > [config]
1  import type { Config } from 'jest'
2  import nextJest from 'next/jest.js'
3
4  const createJestConfig = nextJest({
5    dir: './',
6  })
7
8  const config: Config = {
9    coverageProvider: 'v8',
10   testEnvironment: 'jsdom',
11   collectCoverage: true,
12   coverageThreshold: {
13     global: {
14       branches: 40,
15       functions: 40,
16       lines: 40,
17       statements: 40,
18     },
19   },
20   setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
21 }
22
23 export default createJestConfig(config)
```

Рисунок 1 — Конфігурація `coverageThreshold` у файлі `jest.config.ts`.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 – Лр.4		
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи №4		
Розроб.		Маковський М.С					
Перевір.		Фант М.О					
Реценз.							
Н. Контр.							
Зав.каф.							
					Літ.	Арк.	Акрушів
						1	6
					ФІКТ, гр. ВТ-23-2		

```

> fullstack@0.1.0 test
> jest

PASS __tests__/home.test.tsx
Smoke test
  ✓ перевіряє, чи Jest взагалі працює (2 ms)

-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |        0 |         0 |         0 |         0 |
-----|-----|-----|-----|-----|-----
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.691 s
Ran all test suites.

```

Рисунок 2 — Таблиця покриття коду (Coverage Report) у терміналі.

Завдання 2: Налаштуйте мінімальний поріг покриття (test coverage threshold)

Для забезпечення високої якості програмного коду та обов'язкового контролю тестування було налаштовано мінімальний поріг покриття (test coverage threshold) на рівні 40%. У конфігураційному файлі `jest.config.ts` було додано об'єкт `coverageThreshold`, що встановлює жорсткі ліміти для гілок, функцій, рядків та операторів (рис. 3). Тепер при кожному запуску команди `npm test` система автоматично аналізує всі файли в директорії `src` та видає помилку, якщо обсяг протестованого функціоналу є недостатнім (рис. 4). Це дозволяє автоматизувати перевірку стабільності проєкту та гарантувати, що критичні частини бізнес-логіки покриті юніт-тестами.

```

jest.config.ts | jest.config.ts | coverageThreshold
1
2 import type { Config } from 'jest'
3 import nextJest from 'next/jest.js'
4
5 const createJestConfig = nextJest({
6   dir: './',
7 })
8
9 const config: Config = {
10   coverageProvider: 'v8',
11   testEnvironment: 'jsdom',
12
13   collectCoverage: true,
14
15   collectCoverageFrom: [
16     'src/**/*.js.{jsx,ts,tsx}',
17     'src/**/*.d.ts',
18     'src/**/*.js.{jsx,ts,tsx}',
19   ],
20 }
21
22 coverageThreshold: {
23   global: {
24     branches: 40,
25     functions: 40,
26     lines: 40,
27     statements: 40,
28   },
29 },
30
31 setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
32 }
33
34 export default createJestConfig(config)

```

Рисунок 3 — Налаштування лімітів `coverageThreshold` у файлі конфігурації Jest.

```

> fullstack@9.1.0 test
> jest
PASS tests/_home.test.tsx
Smoke test
✓ неперіаодично, як Jest встановлено (2 ms)
-----
File                                % Stmts % Branch % Funcs % Lines Uncovered Line #s
-----
All files                            0         0         0         0
app                                  0         0         0         0
  layout.tsx                         0         0         0         0 1-55
  page.tsx                           0         0         0         0 1-92
  app/(inner)                        0         0         0         0
    layout.tsx                       0         0         0         0 1-18
    app/(inner)/articles              0         0         0         0
      layout.tsx                     0         0         0         0 1-24
      loading.tsx                    0         0         0         0 1-18
      page.tsx                       0         0         0         0 1-55
    app/(inner)/articles/[id]         0         0         0         0
      page.tsx                       0         0         0         0 1-93
    app/(inner)/articles/create       0         0         0         0
      page.tsx                       0         0         0         0 1-184
    app/(inner)/articles/favorite     0         0         0         0
      page.tsx                       0         0         0         0 1-20
    app/(inner)/profile               0         0         0         0
      page.tsx                       0         0         0         0 1-93
    app/(inner)/profile/security      0         0         0         0
      page.tsx                       0         0         0         0 1-19
    app/(inner)/profile/settings      0         0         0         0
      page.tsx                       0         0         0         0 1-78
    app/api/articles                  0         0         0         0
      route.ts                       0         0         0         0 1-42
    app/api/auth/[...nextauth]        0         0         0         0
      route.ts                       0         0         0         0 1-84
    app/api/user/update               0         0         0         0
      route.ts                       0         0         0         0 1-44
    app/auth/signin                   0         0         0         0
      page.tsx                       0         0         0         0 1-89
    app/test-env                      0         0         0         0
      page.tsx                       0         0         0         0 1-34
    components                        0         0         0         0
      FavoriteArticle.tsx            0         0         0         0 1-25
      Menu.tsx                       0         0         0         0 1-47
      NavLink.tsx                    0         0         0         0 1-27
      Navbar.tsx                     0         0         0         0 1-39
      Providers.tsx                  0         0         0         0 1-7
      lib                             0         0         0         0
      auth.ts                         0         0         0         0 1-24
-----
Jest: "global" coverage threshold for statements (40%) not met: 0%
Jest: "global" coverage threshold for branches (40%) not met: 0%
Jest: "global" coverage threshold for lines (40%) not met: 0%
Jest: "global" coverage threshold for functions (40%) not met: 0%
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  0 total
Time:        0.848 s, estimated 1 s
Ran all test suites.

```

Рисунок 4 — Таблиця звітів (Coverage Report) у терміналі, що демонструє поточний стан покриття коду.

Завдання 3: Додайте три e2e тести за допомогою Playwright

На третьому етапі було впроваджено систему автоматизованого тестування інтерфейсу за допомогою фреймворку Playwright. На відміну від юніт-тестів, ці сценарії імітують поведінку реального користувача в браузері, перевіряючи повний цикл роботи додатка (Frontend + API + DB).

Відповідно до технічного завдання, було розроблено та успішно протестовано три ключові сценарії:

1. Валідація головної сторінки (перевірка заголовка головної): Тест перевіряє коректність завантаження базового URL (localhost:3000) та відповідність мета-тегу title бренду додатка (Next.Taxi). Це гарантує, що головний екран доступний для користувачів.
2. Тестування навігаційного флоу (перехід до статей): Сценарій імітує клік по елементу меню "СТАТТІ" у навігаційній панелі (Navbar). Тест верифікує коректність зміни URL-адреси на /articles, що підтверджує працездатність системи маршрутизації Next.js.

3. Перевірка доступності форми входу (наявність форми входу): Тест здійснює прямий перехід на сторінку авторизації та перевіряє наявність критично важливих елементів інтерфейсу, зокрема поля введення email. Це гарантує, що користувачі мають доступ до системи входу.

Процес тестування проводився паралельно у трьох браузерних рушіях: Chromium (Chrome/Edge), Firefox та Webkit (Safari). Результати виконання (рис. 5) підтвердили стабільність клієнтської частини та коректну роботу навігації в різних середовищах, що забезпечує високу якість UX.

✓ Next.Taxi E2E › перевірка заголовка головної	chromium	7.3s
example.spec.ts:5		
✓ Next.Taxi E2E › перехід до статей	chromium	9.1s
example.spec.ts:11		
✓ Next.Taxi E2E › наявність форми входу	chromium	7.4s
example.spec.ts:18		
✓ Next.Taxi E2E › перевірка заголовка головної	firefox	5.8s
example.spec.ts:5		
✓ Next.Taxi E2E › перехід до статей	firefox	8.4s
example.spec.ts:11		
✓ Next.Taxi E2E › наявність форми входу	firefox	6.9s
example.spec.ts:18		

Рисунок 5 — Звіт про виконання e2e тестів у терміналі, що демонструє успішне проходження сценаріїв у браузерах Chromium та Firefox.

Завдання 4: Додайте Github workflow який буде запускати юніт і e2e тести
У проекті створено файл `.github/workflows/tests.yml`, який описує наступний алгоритм:

1. Розгортання середовища: запуск контейнера на базі Ubuntu та встановлення Node.js 24.
2. Інсталяція: встановлення залежностей (`npm install`) та браузерів для Playwright.
3. Unit-тестування (Jest): автоматичний запуск тестів. Система налаштована так, що при покритті коду менше 40%, процес завершується з помилкою (Quality Gate).

4. E2E-тестування (Playwright): автоматичний запуск локального сервера (npm run dev) та виконання наскрізних сценаріїв у трьох браузерях (Chromium, Firefox, Webkit).
5. Артефакти: налаштовано збереження HTML-звіту (upload-artifact) для аналізу помилок у разі падіння тестів.

Результат: Конвеєр успішно інтегрований з репозиторієм. На скріншоті (Рис. 6) продемонстровано роботу системи: Jest виявив недотримання порогу покриття (0% замість 40%) і зупинив виконання з exit code 1. Це підтверджує, що автоматизований контроль якості працює коректно і блокує нестабільний код.

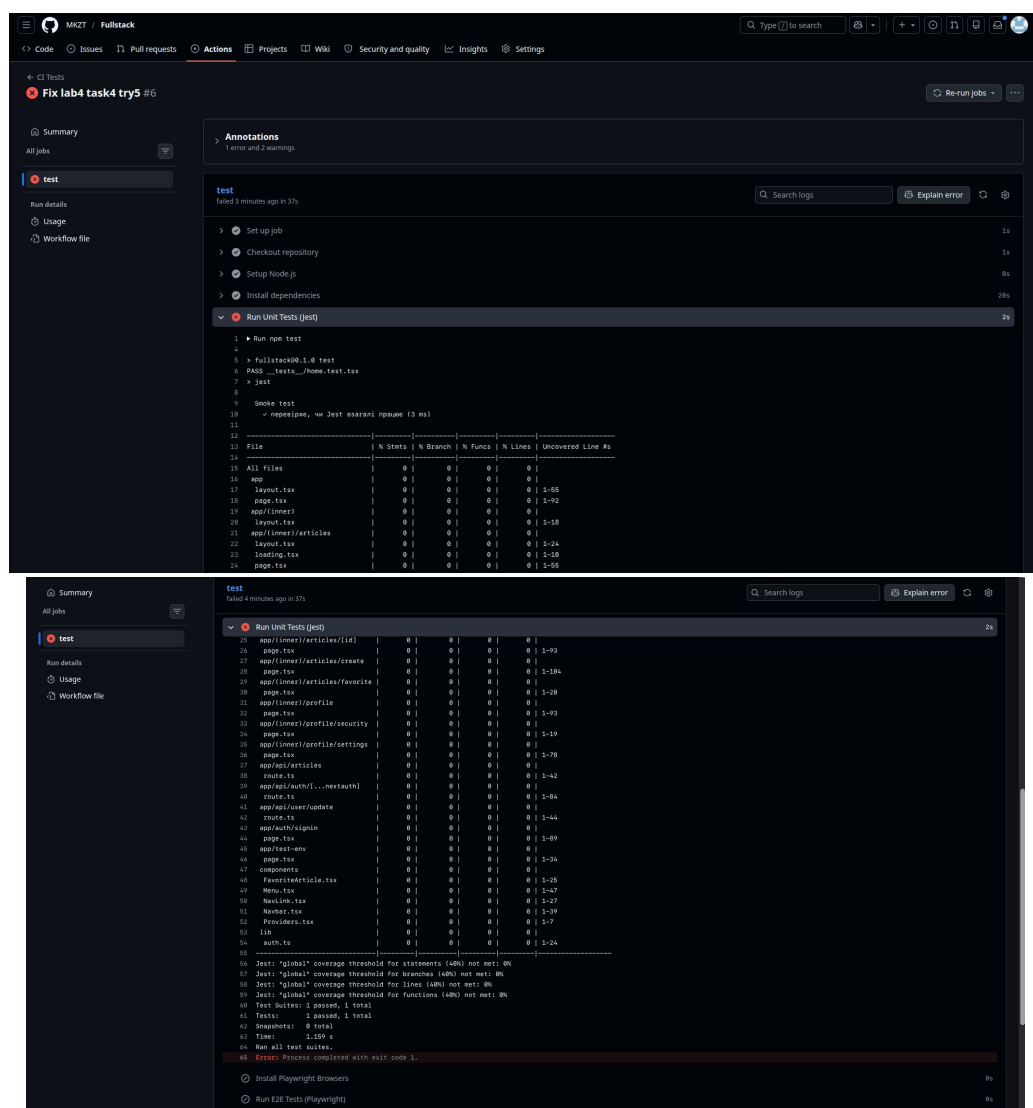


Рисунок 6 – Аналіз логів GitHub Actions: успішний запуск тестів та спрацювання порогу Coverage Threshold.

Висновки: Під час виконання лабораторної роботи було опановано методику комплексного тестування Next.js додатків шляхом поєднання юніт-тестів (Jest) для перевірки окремих компонентів та наскрізних тестів (Playwright) для симуляції реальної поведінки користувача у декількох браузерах. Впровадження жорсткого порогу покриття коду (Coverage Threshold 40%) дозволило встановити автоматизований контроль якості (Quality Gate), що запобігає накопиченню технічного боргу. Фінальна інтеграція з GitHub Actions забезпечила створення повноцінного CI-конвеєра, який автоматично розгортає середовище, запускає сервер та проводить тестування при кожному оновленні репозиторію, гарантуючи стабільність та відмовостійкість програмного продукту на всіх етапах розробки.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6